Verge Systems

an edge in **systems development**

FIRST WEEKLY INTERNSHIP REPORT

WEEK NO 1 (27th - 31st July, 2026)

SUBMITTED TO:
Management of Verge System

SUBMITTED BY:
Hasnain Aziz

DATE OF SUBMISSION:
4th August, 2026

WEB DEVELOPMENT | MODERN SOLUTIONS | RESPONSIVE DESIGN | CLEAN CODE

ACRONYMS

AI	ARTIFICIAL INTELLIGENCE
API	Application Programming Interface
CLI	Central Processing Unit
CSV	Comma-Separated Values
GIT	Git Version Control System (Git is a name, not an acronym)
GITHUB	GitHub (Cloud-Based Version Control and Code Hosting Platform)
GPU	Graphics Processing Unit
HF	Hugging Face
HTML5	Hyper Text Markup Language Version 5
HTTPS	Hyper Text Transfer Protocol Secure
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PYTORCH	PyTorch (Open-Source Machine Learning Framework — not an acronym)
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer
SQL	Structured Query Language
SQLITE	SQLite (Lightweight Embedded Database Engine — not an acronym)
UI	User Interface
UX	User Experience

Table of Contents

Table of Figures

Table of Tables

1. INTRODUCTION

1.1 Internship Overview

This report presents an overview of my internship experience as an **AI and Software Development Intern** at Verge Systems. The internship gave me the opportunity to move beyond theoretical coursework and work hands-on in a professional environment, building and hardening a real, functioning AI application from the ground up.

Over the course of the internship, I worked on an AI-powered multilingual healthcare chatbot (SehatAI), using technologies such as Python, PyTorch, Hugging Face Transformers, Sentence Transformers, and ChromaDB. I learned how to integrate a local language model with a retrieval-augmented generation (RAG) pipeline, design layered safety and scope-restriction systems, build structured evaluation frameworks to systematically test AI behavior, and refactor application architecture to support real-world, multi-user deployment. Working through these tasks deepened my understanding of industry practices, evidence-based debugging, and the full lifecycle of taking an AI prototype toward something production-ready. The internship also strengthened my problem-solving discipline, communication, and ability to work through open-ended technical challenges — skills I expect to carry directly into a future career in AI and software development.

1.2 Purpose of the Internship

The purpose of this internship is to enhance my practical knowledge of web development by The purpose of this internship is to build practical, applied experience in AI and software development by working on a real, evolving project rather than isolated academic exercises. It offers direct insight into how modern AI-powered applications are designed, built, evaluated, and iteratively hardened for real-world use.

The internship also focuses on strengthening both technical and analytical skills — reasoning about model behavior, retrieval accuracy, and system safety — while building fluency with the tools and frameworks used in professional AI development. Through hands-on, project-based work, I developed greater confidence in writing reliable code, diagnosing problems using real evidence rather than assumption, and following professional software engineering practices from design through testing.

1.3 About Verge Systems

Verge Systems is a leading SaaS-based software company headquartered in Hyderabad, Pakistan, with additional offices in the United States, United Arab Emirates, and Saudi Arabia. The company specializes in building innovative software solutions, web and cloud-based products, and IT consulting services for businesses worldwide.

Figure 1:Verge System

Verge Systems supports thousands of clients across more than 200 countries with reliable, scalable, and cost-effective technology solutions. The company places a strong emphasis on innovation, quality, and customer satisfaction, applying modern development technologies and industry best practices across its product line. For interns, Verge Systems offers a genuinely hands-on learning environment, giving them real exposure to professional software and AI development work rather than purely observational experience.

1.3.1 Vision & Mission

Verge Systems' vision is to deliver innovative, dependable, and user-focused technology solutions that help businesses build a strong, effective digital presence. The company aims to create applications that improve user experience, streamline operations, and support sustainable digital growth for its clients.

Its mission centers on developing secure, high-quality, and scalable applications using modern technologies and rigorous development practices, while ensuring the solutions delivered genuinely meet client needs in terms of performance and usability. By fostering a culture of continuous learning, collaboration, and innovation, Verge Systems enables its developers — including interns — to grow their technical expertise while contributing to solutions that create real value for clients around the world.

1.3.2 Core Products & Web Solutions

Verge Systems builds a range of modern, cloud-based applications that help organizations manage their day-to-day operations through secure, user-friendly digital platforms, designed to improve productivity, communication, and overall business performance.

The company's major products include:

- **Web HR** – A cloud-based Human Resource Management System (HRMS) enabling organizations to manage employee records, attendance, leave requests, payroll, recruitment, and performance tracking through a secure web application.
- **Smart Pupils** – A school management platform connecting administrators, teachers, students, and parents, offering attendance tracking, academic records, online communication, and performance monitoring.
- **Financially** – An online financial management system that helps businesses handle accounting, generate financial reports, monitor expenses, and manage transactions from a centralized platform.
- **Pay Day** – A payroll automation application that calculates salaries, manages employee payments, and generates payroll reports with accuracy and efficiency.
- **Hire Side** – A recruitment management platform that streamlines hiring by allowing companies to post openings, receive applications, evaluate candidates, schedule interviews, and manage the full recruitment workflow online.

These products demonstrate Verge Systems' expertise in developing scalable, responsive, and cloud-based web applications. They also reflect the company's commitment to building reliable digital solutions that improve business operations and provide users with an efficient online experience.

Product	Website Domain	Web Development Perspective	Technologies & Web Features
Smart Pupils	smartpupils.com	Educational web portal that provides online access to academic information and communication.	Responsive design, student & teacher portals, online forms, notifications, and database connectivity.
Financially	financially.co	Business web application for managing financial data through an online interface.	Dynamic web pages, secure user accounts, real-time reporting, data visualization, and cloud accessibility.
Pay day	Pay day	Payroll web platform for managing employee salary records online.	Secure login, automated calculations, responsive dashboard, report generation, and database management.
Hire Side	hireside.com	Recruitment web application for handling online hiring activities.	Job listing pages, application forms, candidate dashboard, search filters, and responsive user interface.

Table 1:Products of Verge Systems

1.3.3 Services Offered

Verge Systems provides a broad range of IT solutions designed to help organizations improve their operations and adopt modern digital technology, with services aimed at increasing efficiency, productivity, and long-term growth.

1.3.4 Software as a Service (SaaS)

Verge Systems offers cloud-based software that clients can access over the internet without the need to install or maintain complex on-premises infrastructure. This model provides flexibility, cost savings, automatic updates, and secure access from anywhere with an internet connection.

1.3.5 Custom Software Development

The company builds tailored software solutions based on each client's specific requirements, including business management systems, CRM platforms, workflow automation tools, and other enterprise applications — each developed with the goal of improving operational efficiency and meeting the client's particular objectives.

1.3.6 Information Technology Consultancy

Verge Systems provides professional IT consulting to help organizations plan, implement, and manage technology-driven initiatives. Its consultants assess business needs and recommend appropriate solutions to support digital transformation, including:

- Cloud infrastructure planning and migration
- Cybersecurity evaluation and protection strategies
- Business process analysis and digital transformation planning
- E-governance and public sector technology solutions
- E-commerce planning and online business consultation
- Technology adoption and business process improvement

1.3.7 Website Development Services

Verge Systems builds professional websites for businesses, educational institutions, and other organizations, focused on responsive, attractive, and reliable performance across devices. Services in this area include:

- Custom website design and development
- Domain registration and hosting assistance
- Mobile-responsive, cross-browser compatible websites
- Content Management System (CMS) implementation
- Search Engine Optimization (SEO)
- Ongoing website maintenance, updates, and technical support

Through these services, Verge Systems helps organizations build a strong online presence and connect more effectively with their customers.

1.4 Web Development Technologies

Web development remains a core area of expertise at Verge Systems, which relies on modern technologies and development practices to build secure, responsive, and user-friendly applications across its product line.

1.4.1 Front-End Development

Verge Systems builds modern interfaces using HTML5, CSS3, JavaScript, and Bootstrap, producing responsive layouts and interactive features that provide a smooth experience across desktop, tablet, and mobile devices.

1.4.2 Back-End Development

The company's back-end systems are built primarily with PHP and the Laravel framework, handling server-side logic, authentication, form processing, and application logic to keep applications running efficiently and securely.

1.4.3 Database Integration

MySQL databases are integrated throughout Verge Systems' applications to store, retrieve, and update information reliably while maintaining strong performance at scale.

1.4.4 Responsive Web Design

Verge Systems follows responsive design principles so that its applications adapt automatically to different screen sizes and devices, improving usability and accessibility for all users.

1.4.5 Website Testing and Maintenance

Before launch, Verge Systems conducts thorough testing to catch and resolve technical issues, and continues to provide maintenance, security updates, and performance optimization to keep applications reliable over time.

1.5 Market Presence and Client Relationships

Verge Systems has built a strong industry reputation by consistently delivering reliable software and web solutions to clients around the world. Its focus on quality, innovation, and customer service has helped it form long-term relationships with businesses, educational institutions, healthcare organizations, and government agencies.

The company serves thousands of clients across numerous countries, providing digital solutions that support business growth and operational efficiency, and has earned the confidence of organizations across a wide range of industries through its track record on both local and international projects. Verge Systems works closely with each client to understand their specific goals and deliver solutions tailored to their needs, often integrating third-party services and

technologies to help clients connect existing systems, improve data sharing, and automate workflows.

1.6 Awards and Achievements

Verge Systems has been recognized within the software and IT industry for its commitment to innovation, quality, and customer satisfaction, earning a number of national and international awards over the years.

Year	Award / Achievement
2020	Human Resources Software Awards 2020
2019	Gartner's Get App HR Category Leader
2019	Top 10 Applicant Tracking Software
2018	Front Runners for HRIS Management
2018	Gartner's Get App HR Category Leader
2017	P@SHA 15th ICT Awards – Professional Services Winner

Table 2:Major Awards and Achievements of Verge Systems

These recognitions reflect Verge Systems' dedication to providing innovative software products and dependable technology services. The awards acknowledge the company's focus on product quality, customer satisfaction, and continuous improvement, further strengthening its reputation as a trusted software and web development organization.

1.7 Competitive Strengths of Verge Systems

Verge Systems has established a strong reputation by consistently delivering high-quality technology solutions across a range of industries, combining technical expertise, innovative thinking, and a genuine focus on customer needs.

One of the company's key strengths is its ability to serve clients across different regions while maintaining consistently high standards of quality and professionalism. Its development team applies modern languages, frameworks, and tools to build secure, scalable, and user-friendly applications that support real business growth.

The company also prioritizes staying current — adopting new technologies and industry best practices to keep its products efficient, reliable, and future-ready, with an emphasis on continuous learning and ongoing system improvement. Ultimately, its long-term relationships with clients stem from a consistent focus on understanding what each client actually needs, providing timely support, and following through with solutions that work — which is, in my view, the same standard I tried to hold my own work on the SehatAI project to throughout this internship.

2. Learnings from the Relevant Field

2.1 Summary of Tasks Performed

During this week of my internship at Verge Systems, I worked on an AI-powered healthcare chatbot project (SehatAI), focusing on Retrieval-Augmented Generation (RAG), safety and scope restriction, systematic evaluation, and a major architectural refactor to prepare the system for multi-user deployment. I worked with Python, PyTorch, Hugging Face Transformers, Sentence Transformers, and ChromaDB to build a locally-run AI assistant capable of answering healthcare questions grounded in a curated medical knowledge base.

This week gave me hands-on experience with large language model integration, vector embeddings, semantic search, prompt engineering, systematic AI evaluation methodology, and safe software design for production systems.

2.1.1 Retrieval-Augmented Generation (RAG) Integration

I integrated a Retrieval-Augmented Generation pipeline into the existing chatbot, connecting it to a vector database (ChromaDB) built from twelve curated medical reference documents. I implemented a `Retriever` class that embeds incoming user queries using a multilingual Sentence Transformer model and retrieves the most semantically relevant reference chunks before the language model generates its response.

I also designed a context-aware retrieval mechanism so that short follow-up messages (e.g., "explain more," "short it") correctly inherit the topic of the previous exchange rather than being searched in isolation, which otherwise caused irrelevant or unrelated matches. This required carefully separating what the retriever searches for from what gets permanently stored in conversation memory, to avoid reference material accumulating across turns.

2.1.2 Scope Restriction and Safety Layer Design

I designed and implemented a three-layer scope-restriction system to ensure the chatbot only answers healthcare-related questions:

- **Layer 1:** A system prompt instructing the model to stay within a medical persona and decline unrelated topics.
- **Layer 2:** A keyword-based classifier (`scope_guard.py`) that checks whether a user's message is medically relevant before any retrieval or generation occurs, using word-boundary-safe pattern matching to avoid false positives.
- **Layer 3:** A retrieval-confidence threshold that refuses to answer if nothing in the knowledge base is a close enough semantic match, rather than letting the model improvise an ungrounded answer.

2.1.3 Systematic Model Evaluation

I built a structured evaluation framework to test the chatbot's real-world performance rather than relying on informal spot-checks. This included a 25-question evaluation set spanning clean medical questions, follow-up questions, typo'd/misspelled queries, off-topic questions, and medical emergency scenarios (e.g., snake bite, chest pain).

I built an automated evaluation runner that executes every test question against the chatbot with and without RAG enabled, logs the retrieved reference chunks and their similarity scores, and produces a structured transcript for manual review. Running this evaluation surfaced several real issues, including instances where the model stated specific medical numbers (such as dosages) that were not actually present in the retrieved reference material — a genuine safety-relevant finding.

2.1.4 Debugging, Prompt Engineering, and Fixes

Based on the evaluation results, I diagnosed and fixed several concrete issues:

- Strengthened the prompt design to explicitly instruct the model never to state specific numeric doses, ages, or timing schedules unless they are directly present in retrieved reference material — addressing a hallucination risk found in testing.
- Refined the scope-restriction keyword list to fix both false positives (e.g., "cache" incorrectly matching "ache") and false negatives (plain-language symptom descriptions like "I feel tired and thirsty" not being recognized as medical).
- Fixed a retrieval-context bug where follow-up questions phrased as requests for elaboration ("explain more," "can you elaborate") were incorrectly treated as off-topic due to being absent from the recognized follow-up vocabulary.
- Investigated an intermittent retrieval-mismatch issue using targeted debug logging (query text, matched topics, and raw similarity distances) rather than guessing at fixes without evidence — reinforcing the importance of data-driven debugging over trial-and-error.

2.1.5 Conversation State Refactor for Multi-User Readiness

As the project shifted from a single-user command-line tool toward a planned multi-user web platform, I refactored the chatbot's core architecture to decouple conversation history from the chatbot instance itself. Previously, conversation state was stored as an internal attribute of the chatbot object — a design that would cause simultaneous users to silently share and corrupt each other's conversations in a web environment.

I restructured the response-generation method to accept conversation history as an explicit parameter and return the updated history, rather than mutating shared internal state. I verified this change using isolated test cases confirming that multiple simultaneous conversations remain fully independent, and that the original conversation list passed in by a caller is never

unintentionally modified — a critical correctness guarantee before building any user-facing web layer on top of it.

2.2 Topics Learned

SR. NO.	TOPIC-TECHNOLOGY LEARNED	DESCRIPTION
1	Retrieval-Augmented Generation (RAG)	Generation (RAG)Combining semantic search with language model generation to ground AI responses in trusted source material
2	Vector Embeddings	Representing text as numerical vectors to enable meaning-based (not keyword-based) search
3	ChromaDB	Persistent vector database used to store and query document embeddings
4	Sentence Transformers	Multilingual embedding model used for both document indexing and query search
5	HuggingFace Transformers / PyTorch	Framework used to load and run the underlying language model (Qwen2.5)
6	Prompt Engineering	Designing and testing instructions to control model behavior and reduce hallucination
7	AI-Evaluation Methodology	Structured, evidence-based testing of AI system outputs across varied real-world scenarios
8	Software-Architecture Refactoring	Decoupling shared state to support safe multi-user, concurrent usage
9	Debugging-with Instrumentation	Using targeted logging/debug output to diagnose issues with real evidence instead of guesswork

Table 3:Topics Learned

2.3 Tasks / Assignments Completed

SR. NO.	TASK / ASSIGNMENT	STATUS
1	Integrate a working RAG pipeline	Completed
2	Implemented scope-restriction and safety system	Completed
3	Ran 25-question evaluation framework	Completed
4	Identified medical-dosage hallucination risk	Completed
5	Fixed scope-classification accuracy issues	Completed
6	Refactored core chatbot architecture to support multiple simultaneous users	Completed

Table 4:Tasks Completed

2.4 Solutions Applied / Support Received

When the evaluation results revealed inconsistent behavior in edge cases, I avoided guessing at fixes and instead added targeted debug instrumentation to observe actual system behavior (retrieval queries, matched documents, similarity scores) before making changes. This evidence-first approach helped correctly diagnose issues that an assumption-based fix would likely have missed or only partially solved.

2.5 Key Achievements

- Delivered a functioning, safety-restricted, RAG-grounded healthcare chatbot
- Proactively identified a real hallucination safety issue through systematic testing rather than by accident
- Successfully refactored a single-user prototype into an architecture ready for multi-user deployment, with verified correctness through isolated testing

2.6 Goals for Next Week

- Design and implement the database layer (user accounts, health profiles, chat history persistence)
- Build authentication for the web platform
- Begin wiring the refactored chatbot into a FastAPI backend
- Re-run the full evaluation suite against the refactored system to confirm no regressions were introduced.

3. Technical & Industry Learnings

3.1 Microsoft Launches Its First Cybersecurity AI Model and Agentic Security Platform

During this week, I studied a technology news article about Microsoft's latest advancements in artificial intelligence for cybersecurity. Microsoft introduced its first cybersecurity-specialized AI model, **MAI-Cyber-1-Flash**, along with a new AI-powered cybersecurity platform called **Perception.** These innovations are designed to help organizations detect software vulnerabilities, automate security operations, and strengthen cyber defenses against increasingly sophisticated AI-powered cyberattacks. The launch demonstrates Microsoft's commitment to improving cybersecurity through advanced artificial intelligence.

Figure 2: Microsoft Launches Its First Cybersecurity AI Model

As an AI and software development intern, this article connected directly to the safety and evaluation work I did this week. Building the layered scope-restriction system for the healthcare chatbot — and then systematically testing it with a structured evaluation set to find real weaknesses before they became real problems — follows the same underlying principle as Microsoft's Red/Blue/Green Team model: proactively probing a system for its failure points is what actually makes it trustworthy, not just assuming it works. Reading about automated vulnerability detection also reinforced why I treated my own evaluation findings (like the dosage-hallucination issue) as seriously as a real security flaw, rather than a minor inconvenience to fix later.

This learning experience encouraged me to think about AI safety and reliability not as a one-time checklist, but as an ongoing process of testing, discovering weaknesses, and fixing them — the same mindset now being automated at scale in enterprise cybersecurity. It reinforced the idea that as AI systems take on more responsibility, systematic evaluation and defense-in-depth design, whether in cybersecurity or in a healthcare chatbot, are becoming a core professional skill rather than an optional add-on.

3. The Shift Toward AI-Generated Search and Its Impact on Content Creators

During my internship, I read an article reporting on data showing that Google's AI Overviews — the AI-generated summaries now appearing above traditional search results — went from appearing in roughly fifteen percent of searches to over forty percent within about a year, while visits to Google's more conversational AI Mode nearly tripled over the same period. The broader trend described is a shift from Google as a directory of links toward Google as a destination in itself, where an increasing share of queries are answered directly on the results page rather than through a click-through to an external website.

The article also covered the effect on publishers, who are losing referral traffic as AI-generated answers reduce the need to click through, even as the share of AI responses that include a citation has grown substantially. It noted that some publishers have started blocking AI crawlers unless the crawling company pays for access, effectively treating their own content as licensed material rather than freely indexable text.

As an AI and software development intern, this article was directly relevant to the knowledge-base work I did earlier in this project, where I sourced and adapted educational health content from organizations like WHO, MedlinePlus, and CDC to build SehatAI's reference documents. Reading about the tension between AI convenience for end users and sustainability for the creators of the underlying content reinforced why I treated that sourcing step carefully — paraphrasing rather than copying text directly, and being deliberate about attribution — rather than assuming publicly visible content is simply free to reuse. It also highlighted a real long-term consideration for any AI system that depends on external content: if the sources such systems rely on become harder for creators to sustain, the quality and availability of trustworthy source material could decline over time.

This learning experience encouraged me to think about content sourcing and licensing as a genuine engineering and ethical responsibility, not just a legal formality. It reinforced the idea that as AI systems increasingly mediate how people access information, respecting the sustainability of original content creators is essential to keeping the underlying information ecosystem — and by extension, the AI systems built on top of it — trustworthy in the long run.

3.3 Digital Payments and Opportunities for Pakistan's Digital Economy

During my internship, I studied a newspaper article discussing the challenges faced by Pakistani digital content creators and freelancers due to the limited availability of international online payment services such as PayPal. The article explains that many creators, software developers, and freelancers experience difficulties in receiving payments from global clients because Pakistan does not have direct access to certain international payment platforms. It emphasizes the need for better financial infrastructure and digital payment solutions to support the country's growing digital economy.

Figure 3: Digital Payments

As a web development intern, this article helped me understand the importance of secure online payment systems in the technology industry. Web developers often create websites, e-commerce platforms, and digital services that require reliable payment gateways for online transactions. Learning about these challenges increased my awareness of how financial technology influences freelancing, software development, and digital business opportunities.

This learning experience encouraged me to continue improving my technical skills while understanding the broader role of digital payment systems in the global technology market. It reinforced the idea that innovation, modern financial services, and digital transformation are

essential for supporting web developers, freelancers, entrepreneurs, and the overall growth of Pakistan's IT industry.

3.4 Empty Corner

During my internship, I read a newspaper story titled **"The Empty Corner,"** written by Lara Waseem. The story follows a young girl named Riyam, who deeply misses her late father. Before an important Taekwondo championship, she looks at an old photograph of herself with him and remembers the happy moments they shared, feeling the ache of not having him there to celebrate her success or support her through difficult moments — a part of the story that reflects how the memory of a loved one stays with us long after they are gone.


Story time

The empty corner

The story then describes a family visit during the summer holidays, when Riyam's uncle and cousins come from the United Kingdom to Pakistan and the family visits a horse riding club together. While riding, Riyam slips from the saddle and injures her ankle, but stays silent despite the pain because she feels embarrassed in front of everyone, until her family rushes to comfort and support her.

Figure 4: Empty Corner

As an AI and software development intern, this story's portrayal of quiet resilience resonated with the kind of patience needed when working through repeated technical setbacks — such as the several rounds of debugging the chatbot's retrieval and scope-detection issues this week, where an initial fix would only partially work and required calmly going back, gathering more evidence, and trying again rather than getting discouraged. Just as Riyam's family stepped in to support her, I found that being honest about what wasn't working yet, rather than hiding a setback out of embarrassment, was what actually allowed each issue to get properly resolved.

This learning experience encouraged me to value patience, honesty about setbacks, and steady persistence when facing repeated challenges, whether emotional or technical. It reinforced the idea that courage does not mean never feeling difficulty, but facing it with patience and confidence — and that, just as family support helps us through hard moments, being open about a problem rather than pushing through it alone is often what leads to the best outcome.

4. Learnings from Computer Classes

4.1 Introduction

As part of the internship program, I attended a series of computer training sessions this week led by **Ms. Ayesha Jannat**, a Graphic Designing Intern who took on the role of technical instructor for these sessions. The training was structured around building practical, workplace-ready computer skills, with a particular focus on preparing well-organized, professionally formatted documents in Microsoft Word.

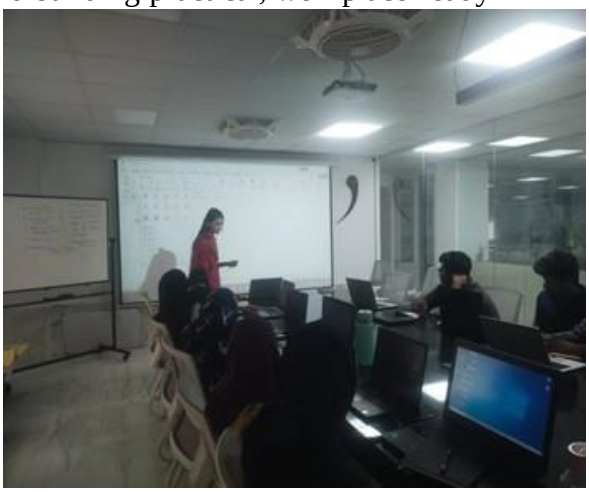

Figure 5:Computer Classes

Rather than covering only basic text editing, the sessions walked through the tools needed to produce polished professional documents — formatting styles, page layout options, structured headings, tables, embedded images, headers and footers, and automatic page numbering. A recurring theme throughout the training was consistency: keeping fonts, spacing, alignment, and overall document structure uniform from start to finish, rather than treating formatting as an afterthought.

Beyond simply learning where each Word feature was located, the sessions reinforced *why* careful formatting matters — that a clean, consistent document is easier to read, presents more professionally, and signals attention to detail to whoever reviews it. Applying these techniques directly while preparing this internship report made the training immediately useful rather than abstract, and it's a skill I expect to rely on for technical documentation well beyond this internship.

4.2 Hardware and Software Fundamentals

As part of the internship's computer training track, I attended a session covering the foundational concepts of computer hardware and software and how they work together within a functioning computer system.

The session walked through the major hardware components that make up a typical machine — the Central Processing Unit (CPU), motherboard, Random Access Memory (RAM), hard disk drives (HDD) and solid-state drives (SSD), along with standard input and output devices such as the monitor, keyboard, mouse, and printer. Beyond simply naming these parts, the training explained how they interact to process, store, and display information, which gave me a clearer mental model of what's actually happening inside a machine while I develop and run software on it.

The session then moved into software concepts, distinguishing system software from application software, and covering operating systems, device drivers, utility programs, and everyday application software such as browsers, office suites, and development tools. The instructor emphasized that hardware and software are fundamentally interdependent — neither is useful without the other.

This grounding in computer fundamentals turned out to be more directly relevant than I expected while working on the AI chatbot project this week, particularly when reasoning about memory usage, device selection (CPU vs. GPU) for running the language model, and general performance troubleshooting. Understanding these fundamentals makes it easier to reason about why a system behaves the way it does, rather than treating the computer as a black box.

4.3 Internship Report Formatting and Documentation Guidelines

The internship also included a set of four sessions focused specifically on Internship Report Formatting and Documentation Guidelines, aimed at helping interns understand the structure, formatting standards, and documentation practices expected in a professional internship report. The training made the point that a well-organized report reflects not just the work completed, but the intern's professionalism, attention to detail, and ability to communicate clearly.

The first session covered the standard structure of an internship report end to end — title page, approval certificate, dedication, acknowledgement, table of contents, list of figures and tables, abbreviations, introduction, company profile, internship activities, technical and industry learning, computer class learning, achievements, conclusion, references, and appendices — along with the purpose each section serves and how they should flow logically from one to the next, written in a clear, formal academic style.

A large portion of the training focused on formatting mechanics in Microsoft Word: setting consistent page margins, selecting an appropriate font such as Times New Roman, applying correct font sizes for headings versus body text, and managing line spacing, paragraph spacing, and alignment. We were shown how using Word's built-in heading styles consistently — rather than manually formatting each heading — both keeps the document looking professional and allows the Table of Contents to be generated and updated automatically.

We also covered the correct use of headings, subheadings, tables, figures, captions, and page numbering, including how to structure hierarchical heading levels, number chapters and subchapters correctly, and generate automatic lists of figures and tables that stay in sync with the document. Alongside this, the training highlighted common formatting mistakes to watch for — inconsistent fonts, incorrect heading numbering, misaligned tables, missing captions — and practical techniques for proofreading a report before submission.

The final session addressed the practical side of submission: converting the finished report to PDF while preserving formatting, double-checking page numbers and figure/table labels, and writing a professional submission email — an appropriate subject line, a respectful and concise message, correctly attached files, and submitting ahead of the deadline. These sessions were genuinely useful beyond just this report; the same formatting and documentation discipline applies directly to technical write-ups, project documentation, and any other professional document I'll need to produce going forward.

5. Intern Feedback

My internship at Verge Systems has been one of the most valuable and rewarding learning experiences of my academic journey so far. It gave me a genuine opportunity to bridge the gap between what I had studied in theory and what it actually takes to build a working AI-powered software system in a professional setting.

Throughout the internship, I worked in a professional environment where I learned not only applied AI and software engineering skills, but also workplace ethics, teamwork, clear communication, and — perhaps most importantly — how to debug real, non-obvious problems methodically rather than by guesswork. This experience has meaningfully strengthened my confidence and technical judgment, and prepared me for the kind of open-ended problem-solving that real software projects actually demand.

From the beginning, I was welcomed into a supportive working environment where I was encouraged to investigate problems myself, propose solutions, and treat every bug or unexpected result as something to understand rather than something to quickly patch over. That mindset shaped how I approached this internship's core project: building and hardening SehatAI, a locally-run, multilingual healthcare chatbot designed for rural communities with limited access to reliable medical information.

One of the most valuable aspects of this internship was working on a genuinely production-shaped AI system rather than an isolated tutorial exercise. Over the course of the project, I worked hands-on with Python, PyTorch, Hugging Face Transformers, Sentence Transformers, and ChromaDB to take a basic local language model (Qwen2.5) and turn it into a chatbot grounded in a curated medical knowledge base through Retrieval-Augmented Generation (RAG). Every new concept — embeddings, vector search, prompt engineering, layered safety design — was something I had to actually apply and then test against real, sometimes messy, real-world input, rather than just read about.

A major achievement during this internship was designing and implementing a three-layer scope-restriction system to keep the chatbot reliably within its intended healthcare domain: a system-prompt-level instruction, a keyword-based classifier layer, and a retrieval-confidence layer that refuses to answer when nothing in the knowledge base is a close enough match. Getting this right required real iteration — my first version had both false positives (rejecting legitimate symptom

descriptions phrased in plain language) and false negatives (letting off-topic questions like "how do I fix my wifi" slip through) — and fixing it taught me a great deal about the gap between a design that looks correct on paper and one that actually holds up against real usage.

Just as significant was building a structured evaluation framework rather than relying on informal testing. I designed a 25-question evaluation set spanning clean medical questions, follow-ups, typos, off-topic questions, and medical emergencies, and built an automated runner to test the chatbot systematically, with and without RAG enabled, logging retrieved reference chunks and similarity scores for review. This process directly surfaced a real safety issue — the model stating a specific medical dosage that wasn't actually present in its reference material — which I then diagnosed and fixed. Finding and fixing that issue through systematic testing, rather than by chance, was one of the most professionally meaningful moments of the internship; it made clear why rigorous evaluation isn't optional for anything AI-driven that real people might rely on.

Later in the internship, as the project's direction shifted from a single-user command-line tool toward a planned multi-user web platform, I refactored the chatbot's core architecture to decouple conversation history from the chatbot instance itself — a necessary change, since the original design would have caused simultaneous users to silently share and corrupt each other's conversations. I verified the refactor with isolated test cases before considering it complete, rather than assuming it worked because the code looked right.

Throughout the internship, debugging was rarely a one-shot process. Several issues required multiple rounds of hypothesis, targeted debug logging, and re-testing before I actually found the real cause rather than a plausible-sounding one. I learned to resist the temptation to patch a symptom and move on, and instead to gather real evidence — actual similarity scores, actual retrieved content, actual model output — before deciding a fix was correct. This iterative, evidence-first approach to debugging is something I expect to carry into every technical project I work on going forward.

Alongside the technical work, the computer classes and industry-awareness sessions organized during the internship broadened my perspective on professionalism, ethics, and the wider technology industry — from AI-driven cybersecurity automation to how AI-generated search results are reshaping the sustainability of the content creators such systems ultimately depend on. These discussions reinforced that being a good AI/software developer requires more than technical skill; it requires integrity, an awareness of who is affected by what you build, and a genuine commitment to getting things right rather than just getting them to appear to work.

The guidance from my supervisors was one of the most valuable parts of this experience. Rather than handing me solutions when something broke, they consistently pushed me to investigate the problem myself, gather evidence, and reason through it — an approach that noticeably strengthened my independent debugging and problem-solving ability over the course of the internship.

If I could suggest one improvement to the program, it would be additional structured exposure to deploying AI systems in production — authentication, database design for multi-user

applications, and API design — since the project I worked on is heading in exactly that direction, and having more structured guidance on that transition would be valuable for future interns doing similar work.

Overall, this internship has been an outstanding learning experience that meaningfully strengthened both my technical and professional capabilities. It gave me real, hands-on experience building and hardening an AI system end-to-end, while sharpening my analytical thinking, debugging discipline, and communication skills. I sincerely appreciate the continuous guidance, patience, and support provided by my supervisors and the entire Verge Systems team throughout this process.

6. Conclusion

The internship at Verge Systems has been an enriching and formative experience that significantly contributed to my academic, technical, and professional development. It gave me the opportunity to apply theoretical knowledge to a real, evolving AI software project rather than an isolated academic exercise.

Working in a professional development environment allowed me to gain hands-on experience with the full lifecycle of building an AI system — from initial model integration, through retrieval-augmented generation and safety design, to systematic evaluation and architectural refactoring for production readiness. This internship played a significant role in preparing me for future work in AI and software development by strengthening both my technical ability and my confidence in tackling genuinely open-ended problems.

Over the course of the internship, I gained practical experience with Python, PyTorch, Hugging Face Transformers, Sentence Transformers, and ChromaDB, developing a working understanding of how large language models, vector embeddings, and retrieval systems combine to build a grounded, trustworthy AI application. Designing and implementing a Retrieval-Augmented Generation pipeline was one of the most valuable parts of this experience, as it required understanding not just how to call these tools, but how to reason about failure modes — irrelevant retrieval, hallucinated details, and misclassified queries — and to fix them with evidence rather than assumption.

A major achievement of this internship was the design and implementation of a layered scope-restriction and safety system for the chatbot, paired with a structured, repeatable evaluation framework. Building this system end-to-end — from initial design, through real-world testing, to diagnosing and fixing concrete failures such as a medical-dosage hallucination — gave me practical experience with the complete process of making an AI system not just functional, but genuinely trustworthy. This is, in my view, the most important lesson of the internship: a system that appears to work in casual testing and a system that has been rigorously verified to work are very different things, and the gap between them is where real engineering effort belongs.

In addition to technical training, the internship placed real emphasis on professionalism, ethics, communication, and continuous learning. The industry-awareness sessions and computer classes covered topics ranging from workplace ethics and teamwork to the growing role of AI in cybersecurity and its impact on digital publishing — all of which reinforced that success in this field depends as much on judgment and integrity as it does on technical skill.

Throughout the internship, I received continuous, constructive guidance from my supervisors, who consistently encouraged me to investigate problems independently, back up conclusions with real evidence, and only consider an issue resolved once it had actually been verified — not merely once it looked resolved. This approach shaped how I now default to approaching any technical problem, and it is something I expect to carry forward into future projects well beyond this internship.

Overall, this internship successfully bridged the gap between academic learning and real professional practice. It gave me practical, hands-on experience with modern AI development tools and techniques, along with a much sharper sense of what rigorous, trustworthy engineering actually requires. I am sincerely grateful to my supervisors, mentors, and the entire Verge Systems team for their guidance and support throughout this experience, and I'm confident the skills and habits built during this internship — particularly the discipline of testing assumptions rather than trusting them — will continue to serve me well throughout my academic and professional career.